



Lab Instructions No. 4

Raspberry Pi Pico MicroPython Programming

2026-06-16

Computer Structure Class Spring 2025

Sharif University of Technology

Spring 2026

By Dr. Saeed Bagheri Shouraki



Fig. 1 — Raspberry Pi? Buttons?

I. Introduction

The Raspberry Pi Pico is a microcontroller (μC) development board based on the RP2040 chip. The onboard LED (connected to GPIO 25) and external push buttons are used in this lab to demonstrate GPIO operations and serial interaction.

II. Background

II.1. Interfacing with the Emulator via Serial Terminal

At first you need to sign up for [Wokwi](#).

You will use to interface with your microcontroller. You may check the official micropython documentation at [Quick reference for the RP2](#).

II.2. Hardware Setup Notes

Please DO NOT use the 5V power lines. Also it is best to avoid using GPIO pins 26, 27, 28 as these pins are inherently more vulnerable.

II.3. Debouncing

When a mechanical push button is pressed or released, its internal contacts do not immediately make or break the electrical connection cleanly. Instead, they tend to bounce or vibrate for a brief period, causing rapid fluctuations between open and closed states. This bouncing phenomenon results from the physical movement of the metallic contacts, which oscillate before settling into a steady state. These rapid state changes, known as contact bounce, can last anywhere from a few microseconds to tens of milliseconds depending on the button's design and quality. In digital circuits, this bouncing can be problematic, as it may generate multiple unwanted state changes instead of a single, clean transition, potentially leading to incorrect operation or erratic behavior in connected devices.

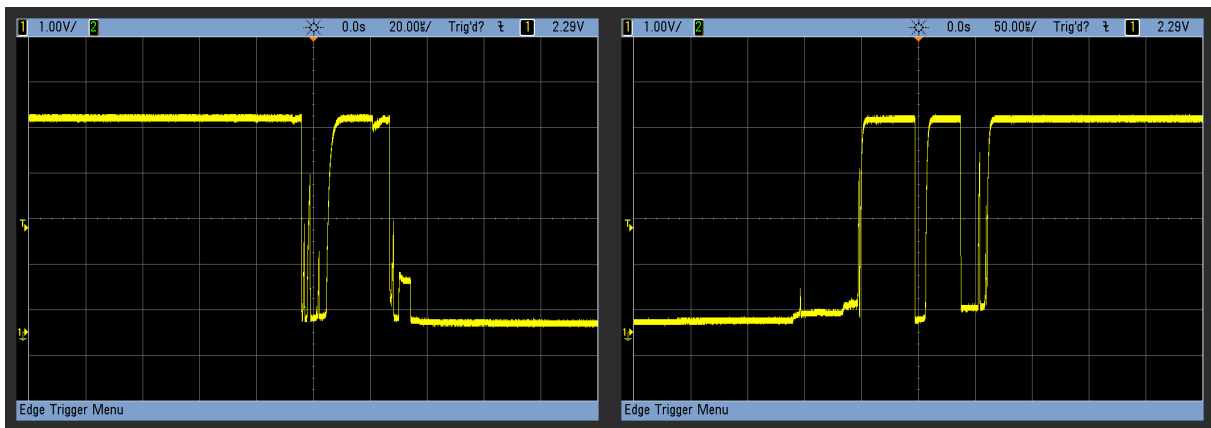


Fig. 2 — A Button Changing State

One common method to mitigate the effects of button bouncing is by introducing a delay after detecting the initial state change. This technique, called debouncing, ensures that any rapid fluctuations occurring within the delay period are ignored, allowing the button contacts to stabilize. By using a short delay -typically in the range of 10 to 50 milliseconds- after a state transition is first detected, the system effectively filters out the noise caused by bouncing.

III. Implementation Guide

Excerpts from this code might be useful:

```
import time

from machine import Pin

# Note: machine library is used to interface the hardware

led = Pin(25, Pin.OUT)
p2 = Pin(2, Pin.IN)

print(time.time()) # Surprise!!!
print(time.time()) # Surprise!!!
```

```
led.on() # Could have used led.value(1)

time.sleep_ms(2000)

led.off() # Could have used led.value(0)

print(time.time()) # Surprise!!!

print(p2.value()) # Read GPIO2

t1 = time.ticks_us()
led.on()
led.off()
t2 = time.ticks_us()

print(time.ticks_diff(t2, t1))
# Never do direct arithmetic with ticks
# always use ticks_diff and ticks_add

my_list = []
for i in range(100):
    my_list.append((i % 7) % 3)

for x in my_list:
    if (x % 2) == 0:
        led.on()
    else:
        led.off()

time.sleep_ms(100)
```

IV. Tasks

IV.1. Adjustable Blink (20 pts)

Write a program to blink the onboard LED (GPIO 25) with periods specified by the user. The user inputs two integers via the serial monitor:

- On time in milliseconds (first number).
- Off time in milliseconds (second number).

IV.2. Adjustable Bouncer (20 pts)

Write a program that implements bounce effect on connected LEDs. The user must be able to specify the bouncing period using the serial terminal.

IV.3. Reading GPIOs (20 pts)

Write a program to make the onboard LED follow the state of a push button connected to a GPIO pin. The button pin should be configured in pull-up mode.

IV.4. Debouncing (40 pts)

Write a program to count the number of state changes (presses and releases) of a push button. Implement a debouncing strategy to filter out spurious transitions caused by mechanical noise.

Use a delay in the range of a 10...50 ms to debounce your button. You may use Logic Analyzer to view and save the samples as a ".vcd" file.